

Assignment 2

System Overview

The goal of this project: given a photo of a car part (headlight, bumper), find the moments in a video where that part is seen. The source video is a ~46-minute 2026 Toyota RAV4 review from YouTube. The query images come from a HuggingFace dataset of RAV4 exterior shots.

I built a five-stage pipeline that downloads the video, samples frames, runs a car-part detector on each frame, indexes the results into a Parquet file, and then matches query images against that index. Everything runs inside the Docker devcontainer with no manual labeling.

Detector

I used YOLOv8n-seg. It's small (3.27M parameters) and fast to process. More importantly, Ultralytics ships a car-parts segmentation dataset with 23 classes like front_bumper, hood, which is the assignment's goal.

I fine-tuned the model for 5 epochs on a Colab T4 GPU. Training took about 7 minutes. On the commonly visible classes the results were decent:

- front_bumper : 0.955
- front_glass : 0.941
- hood : 0.941
- back_bumper : 0.908.

I set the confidence threshold to 0.25, which is lower than the usual 0.5 default. This enables detections in tricky frames, but also results in a few false positives.

Video Sampling

The video was downloaded with yt-dlp, forcing H.264 encoding. I initially tried letting yt-dlp pick the best format, which used an AV1 stream. This caused decoding issues and warning—OpenCV couldn't decode it, and even ffmpeg threw warnings. Switching to H.264 fixed these problems.

I used ffmpeg for frame extraction instead of OpenCV's VideoCapture. This was more reliable and produced 2,794 frames at one frame per second. A car review video doesn't change shots faster than once every few seconds, so I decided to use a 1 FPS rate.

Retrieval Logic

The matching approach is simple. I run the same detector on the query images to extract class labels. Then I look up those labels in the Parquet index. For each matching class, I collect the timestamps of detection in the video and merge consecutive ones into clips.

Merging

- I sort the timestamps for a given class and iterate through them.
- While next timestamp is within 5 seconds of the previous one, the current segment is extended
- If the gap is larger than 5 seconds, that segment is closed and a new one starts.

Each segment includes the start and end timestamps, matched class, number of supporting frame detections, and a YouTube embed URL. Running this against all 65 query images produced 13,467 retrieval results.

Parquet Schema and Data Quality

Column	Type	Description
video_id	string	YouTube ID (YcvECxtXoxQ)
frame_index	int64	ffmpeg frame number, 1-indexed
timestamp_sec	int64	Seconds into the video, 0-indexed
class_label	string	Car part name (e.g. front_bumper)
x_min, y_min	float64	Bounding box top-left
x_max, y_max	float64	Bounding box bottom-right
confidence_score	float64	Model confidence, 0 to 1
detector_name	string	Model file used (best.pt)

The file is sorted by timestamp. The whole file is about 200KB. It's published at huggingface.co/datasets/PrashanthNYU/car-parts-retrieval.

What the Detector Found

The most common detections are :

- back_glass (1,423) and back_light (1,325) a lot of the footage shows the car from behind as it drives away or is parked.
- Hood (667), wheel (644), and mirrors (572 right, 495 left) are also frequent since they're visible in side-profile.
- Parts like front_bumper (387) and front_glass (383) appear less often because the reviewer spends less time filming the car head-on.
- The rarest detections are tailgate (3), front_right_door (2), and front_left_door (11) which are rare in the footage or the model struggles to detect them due to limited training data.

Where Things Go Wrong

About 40% of the video is the interior cabin where no exterior parts are visible. The detector produces zero correct detections here. So a large part of the video is not used correctly by the retrieval system

The more relevant issue is the vocabulary mismatch at the class level. The training dataset contains 23 classes, a query image might depict something the model doesn't have a label for, e.g. a roof rack, a certain trim, or a badge. In such cases, the model outputs nothing or incorrectly classifies the part, resulting in empty or incorrect outputs.

Some classes just don't have enough training data. Tailgate had only 5 training images, and the model only found 3 instances across the entire video. For queries featuring those parts, retrieval coverage will be poor

Conclusion

If we give the system a photo of a car part, and it gives you back timestamped clips from the video where that part appears, with links to verify. The label-based matching is coarse and the model misses some rare parts but it handles the common cases well. The whole pipeline is reproducible from the Docker devcontainer with a single command.